Unité 11 — Fuzzing (AFL++)

Livrable B — Support de cours · UE « Software Security »

Code UE 7532 14 U32 D1 · Contenu auteur · 11 octobre 2026

> **Place dans le programme.** Cette unité répond directement au point du dossier pédagogique « **développer des séquences de test de fuzzing** ». *Le fuzzing n'est qu'évoqué dans le support de référence meltypot (« using tools such as fuzzing… ») : toute cette unité, y compris le laboratoire AFL++, est un contenu **auteur**, construit et vérifié expérimentalement.* Les sorties marquées « réelle » proviennent d'une campagne effectivement exécutée.

1. Qu'est-ce que le fuzzing ?

Le **fuzzing** consiste à exécuter un programme en lui fournissant massivement des entrées automatiquement générées ou mutées, afin de provoquer des comportements anormaux (plantages, dépassements, assertions). C'est aujourd'hui l'une des techniques les plus efficaces pour découvrir des vulnérabilités mémoire dans du code C/C++, précisément parce que les tests classiques couvrent ce que l'utilisateur *fait*, tandis que l'attaquant explore ce qu'il ne ferait *jamais*.

2. Familles de fuzzing

Axe	Distinction
Connaissance de la cible	**boîte noire** (aucune) · **boîte grise** (instrumentation légère, retour de couverture) · **boîte blanche** (analyse/symbolique).
Production des entrées	**mutation** (on part d'un corpus et on le déforme) · **génération** (on fabrique selon une grammaire/format).
Guidage	**« dumb »** (aléatoire pur) · **guidé par la couverture** (on garde les entrées qui atteignent du code nouveau).

AFL++ est un fuzzer **mutation, boîte grise, guidé par la couverture** — le standard pédagogique et industriel.

3. Le fuzzing guidé par la couverture (AFL++)

Principe : on **instrumente** la cible à la compilation pour qu'elle signale, à chaque exécution, quelles arêtes du graphe de flot de contrôle ont été parcourues. AFL++ maintient une **file** (queue) d'entrées intéressantes ; une entrée qui atteint une arête *nouvelle* est conservée et re-mutée. La boucle :

BOUCLE DE FUZZING GUIDÉ PAR LA COUVERTURE

```
corpus initial ──► [ mutation (havoc, splice, …) ] ──► execution instrumentee
      ▲                                                          │
      │              nouvelle couverture ? ── oui ──► ajoutee a la file
      └──────────────────────── non ◄── rejetee
                            crash ? ── oui ──► sauvegardee dans crashes/
```

C'est ce retour de couverture qui permet à AFL++ de « deviner » des conditions difficiles (octets magiques, longueurs) sans connaître le format : franchir une nouvelle branche est récompensé.

4. Pourquoi coupler le fuzzing aux sanitizers

Un bug mémoire ne provoque pas toujours un plantage immédiat (on l'a vu en Unité 2 : un petit dépassement peut passer inaperçu). **AddressSanitizer (ASan)** instrumente les accès mémoire et transforme ces corruptions silencieuses en **abandons immédiats et localisés**. Stratégie recommandée :

- **Binaire de campagne** : instrumenté pour la couverture, rapide (AFL détecte les crashs via SIGSEGV/abandon).
- **Binaire de triage** : compilé avec ASan, pour obtenir un rapport précis (type de bug, fichier, ligne) en rejouant un crash.

5. Anatomie d'une campagne — le workflow complet

Le laboratoire 11 déroule exactement ces étapes :

1. **Préparer la cible** : isoler la fonction à tester, lui fournir une entrée (stdin ou fichier).
2. **Instrumenter** : compiler avec `afl-clang-fast`.
3. **Constituer un corpus** : quelques entrées valides minimales (graines).
4. **Lancer** `afl-fuzz` et **surveiller la couverture**.
5. **Trier** les crashs : reproduire, minimiser, dédupliquer, évaluer l'impact.
6. **Corriger** la vulnérabilité.
7. **Re-fuzzer** pour confirmer que le crash a disparu.

6. Le harnais (harness)

Un **harnais** est le petit programme qui relie l'entrée fournie par le fuzzer à la fonction à tester. Bon harnais : lit l'entrée (stdin/fichier), appelle la fonction cible, se termine proprement, et est **déterministe** (pas d'aléa, pas d'horloge, pas de réseau) pour que la couverture soit stable.

7. Lire l'écran de statut d'AFL++

Indicateurs à surveiller (extrait réel de `fuzzer_stats` d'une campagne de ce labo) :

FUZZER_STATS — CAMPAGNE RÉELLE (CIBLE VULNÉRABLE)

```
run_time       : 22
execs_done     : 189862
execs_per_sec  : 8628.91
corpus_count   : 7
bitmap_cvg     : 50.00%
stability      : 100.00%
saved_crashes  : 1
```

- **execs_per_sec** : vitesse (ici ~8600/s) ; si faible, cible trop lente.
- **bitmap_cvg** : couverture atteinte.
- **stability** : 100 % = exécutions déterministes (indispensable).
- **saved_crashes** : crashs uniques trouvés (ici 1, en ~18 s).
- **corpus_count** : entrées intéressantes conservées.

8. Le triage des crashs

Reproduire l'entrée sur le binaire ASan donne un diagnostic précis :

RAPPORT ASAN RÉEL (REJEU D'UN CRASH DU LABO)

```
==28468==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x7f6b...060
```

```
WRITE of size 133 at 0x7f6b...060 thread T0
    #0 0x5623...0d7 in __asan_memcpy
    #1 0x5623...989 in parse /data/vuln.c:13:9
Address 0x7f6b...060 is located in stack of thread T0 at offset 96 in frame
    #0 0x5623...73f in parse /data/vuln.c:9
SUMMARY: AddressSanitizer: stack-buffer-overflow in __asan_memcpy
```

On lit : un **stack-buffer-overflow**, écriture de **133 octets** dans un cadre où le tampon n'en fait que 64, à la **ligne 13** de `vuln.c` (le `memcpy`). **Minimiser** (`afl-tmin`) réduit l'entrée à l'essentiel : dans ce labo, de **137 à 87 octets**. **Dédupliquer** : regrouper les crashs par trace/arête pour ne pas traiter dix fois le même bug. **Évaluer l'impact** : écriture hors-bornes contrôlée → potentiellement exécution de code (cf. Unité 2) → gravité élevée (cf. CVSS, Unité 10).

9. Bonnes pratiques et limites

- Corpus de départ petit mais varié ; dictionnaire (`-x`) pour les formats à mots-clés.
- Toujours re-fuzzer après correctif (test de non-régression de sécurité).
- Limites : le fuzzing prouve la présence de bugs, jamais leur absence ; il peine sur les conditions très improbables (checksums, chiffrement) sans aide (`cmplog`, dictionnaires, harnais dédiés).

10. Points clés

- Le fuzzing guidé par la couverture explore automatiquement le code en récompensant les nouvelles branches.
- Coupler campagne (rapide) et triage (ASan) donne des diagnostics précis.
- Une campagne va de la préparation de la cible au re-fuzz de validation du correctif.
- Répond directement à l'exigence du dossier : « développer des séquences de test de fuzzing ».

Contenu auteur (le fuzzing n'est pas développé dans meltypot). Cible, campagne, crash, rapport ASan et correctif ont été produits et vérifiés dans l'environnement de référence (AFL++ 4.09c, clang/LLVM 17).